

Experiment 'p_4_1_e_sum_odd_digits' Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_4_1_e_sum_odd_digits

Natural language statement: Write a method with loops that computes the sum of all odd digits of an input. (For example, if the input is 32677, the sum would be $3 + 7 + 7 = 17$.)

Method signature: p_4_1_e_sum_odd_digits(a: int) returns (sum: int)

Ensures

- `sum >= 0`
- `sum == s_o_d(a)`

Requires

- `a >= 0`

Functional Code Given

```
function s_o_d(a: int): int
requires a >= 0
{
  if a == 0 then 0
  else if a % 2 == 0 then s_o_d(a / 10)
  else a % 10 + s_o_d(a / 10)
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method with loops that computes the sum of all odd digits of an input. (For
→ example, if the input is 32677, the sum would be $3 + 7 + 7 = 17$.)

The signature should be:

`p_4_1_e_sum_odd_digits(a: int)` returns (sum: int)

The method should respect the following contract:

requires `a >= 0`, ensures `sum >= 0`, ensures `sum == s_o_d(a)`

The contract uses the following dafny code:

```
function s_o_d(a: int): int
requires a >= 0
{
  if a == 0 then 0
  else if a % 2 == 0 then s_o_d(a / 10)
  else a % 10 + s_o_d(a / 10)
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dme3swDYAesF9z9CEgFx10Irq5GkN

Model: gpt-5.5-2026-04-23

Created at: 1780486112

Finish reason: stop

Usage: 518 tokens in, and 2783 tokens out

```
{
  var x := a;
  sum := 0;
  while x > 0
    invariant x >= 0
    invariant sum >= 0
    invariant sum + s_o_d(x) == s_o_d(a)
    decreases x
```

```

{
  if x % 2 == 0 {
    assert s_o_d(x) == s_o_d(x / 10);
    x := x / 10;
  } else {
    assert x % 10 >= 0;
    assert s_o_d(x) == x % 10 + s_o_d(x / 10);
    sum := sum + x % 10;
    x := x / 10;
  }
}
assert x == 0;
assert s_o_d(x) == 0;
}

```

Final Program

```

function s_o_d(a: int): int
requires a >= 0
{
  if a == 0 then 0
  else if a % 2 == 0 then s_o_d(a / 10)
  else a % 10 + s_o_d(a / 10)
}

method p_4_1_e_sum_odd_digits(a: int) returns (sum: int)
requires a >= 0
ensures sum >= 0
ensures sum == s_o_d(a)
{
  var x := a;
  sum := 0;
  while x > 0
    invariant x >= 0
    invariant sum >= 0
    invariant sum + s_o_d(x) == s_o_d(a)
    decreases x
  {
    if x % 2 == 0 {
      assert s_o_d(x) == s_o_d(x / 10);
      x := x / 10;
    } else {
      assert x % 10 >= 0;
      assert s_o_d(x) == x % 10 + s_o_d(x / 10);
      sum := sum + x % 10;
      x := x / 10;
    }
  }
}
assert x == 0;
assert s_o_d(x) == 0;

```

}

Total Token Usage

Input tokens: 518

Output tokens: 2783

Reasoning tokens: 2559

Sum of 'total tokens': 3301

Experiment Timings

Overall Experiment started at 1780486112028, ended at 1780486170026, lasting 57998ms (58.00 seconds)

Iteration #1 started at 1780486112034, ended at 1780486170026, lasting 57992ms (57.99 seconds)